

LECTURE

Using the Terraform CLI





What is the Terraform CLI?

The Terraform CLI is the primary way to interact with Terraform and execute the workflow steps



Unified **Workflow**

The CLI ensures consistency and repeatability across your workflow.
– regardless of what platform that you're using Terraform



Command **Interface**

A command-line tool to manage infrastructure through commands like terraform init, plan, and apply



Consistent **Pattern**

Follows a consistent pattern:
`terraform <command> [options]`

`tofu <command> [options]`



Getting to Know the Terraform CLI

Terraform commands follow this structure:

→ `terraform <subcommand> [options or flags]`

```
terraform  plan  -out=planfile
```

Base command to
invoke the CLI

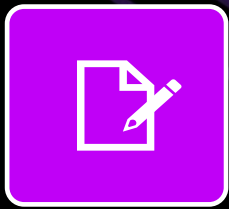
Subcommand

Optional Flag



Mapping the Workflow to Terraform Commands

Write



terraform
fmt

Initialize



terraform
init

Plan



terraform
plan

Apply



terraform
apply

Destroy



terraform
destroy

Using Environment Variables with the CLI



- ❧ Environment variables provide a way to pass configuration settings and credentials to Terraform without hardcoding them into your files.
- ❧ They increase security by keeping sensitive data like API keys out of .tf files.

```

$ export TF_LOG=DEBUG

PowerShell
PS> $Env:TF_LOG = "DEBUG"

Command Prompt
C:\> setx TF_LOG = "DEBUG"

C:\> setx TF_VAR_svr_name = "prod-db-01"

C:\> setx AWS_ACCESS_KEY_ID = "1a2b3c4d"
```



Making the Most of the Terraform CLI



Auto-Complete

Tab completion helps you quickly fill in subcommands, flags, and file paths by pressing the **Tab** key.

It reduces typing errors and ensures you use valid options for Terraform commands.

Available for most shell environments (like Bash or Zsh) after enabling it with `terraform -install-autocomplete`.

```

$ touch ~/.bashrc
$ terraform -install-autocomplete

$ terraform s <tab>
show      state
```




Making the Most of the Terraform CLI

Using the `help` feature

```

$ terraform --help
Usage: terraform [global options] <subcommand> [args]

The available commands for execution are listed below.
The primary workflow commands are given first, followed by
less common or more advanced commands.

Main commands:
  init          Prepare your working directory for other commands
  validate      Check whether the configuration is valid
  plan          Show changes required by the current configuration
  apply         Create or update infrastructure
  destroy       Destroy previously-created infrastructure
  ...

```



Making the Most of the Terraform CLI

Using the **help** feature

TERMINAL

```
$ terraform plan --help
```

```
Usage: terraform [global options] plan [options]
```

Generates a speculative execution plan, showing what actions Terraform would take to apply the current configuration. This command will not actually perform the planned actions.

You can optionally save the plan to a file, which you can then pass to the "apply" command to perform exactly the actions described in the plan.

Plan Customization Options:

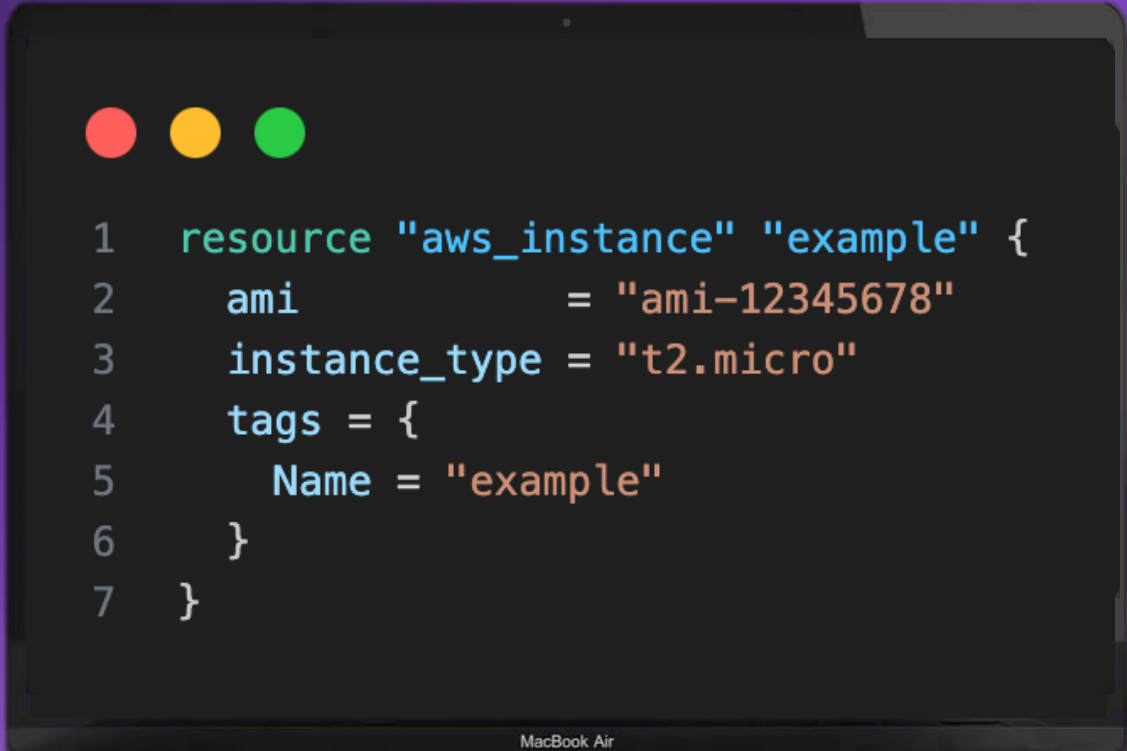
The following options customize how Terraform will produce its plan. You can also use these options when you run "terraform apply" without passing it a saved plan, in order to plan and apply in a single command.

-destroy	Select the "destroy" planning mode, which creates a plan to destroy all objects currently managed by this
----------	---

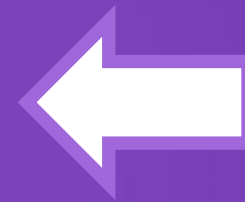


Making the Most of the Terraform CLI

Easily Format Your Code To a Canonical Format and Style



```
1 resource "aws_instance" "example" {
2     ami          = "ami-12345678"
3     instance_type = "t2.micro"
4     tags = {
5         Name = "example"
6     }
7 }
```



`terraform fmt`

Format all Terraform files in
the current directory

Use `-recursive` to format
files in subdirectories as well

Making the Most of the Terraform CLI



Validate Your Configurations

The `terraform validate` command checks your Terraform configuration files for syntax errors and ensures they are structurally valid. It does not check whether the configuration will successfully deploy but ensures your code is properly written and references resources and variables correctly.

This is an essential step to catch errors early before running commands like `plan` or `apply`.

SUMMARY



Use terraform <subcommand> **to manage each step of the workflow**



Enable terraform -install-autocomplete **to speed up typing and reduce errors**



Customize commands with options and flags depending on the subcommand you're using.



Run terraform validate **to catch syntax errors early**



Use terraform fmt **to ensure clean and consistent formatting**



Pass sensitive data and reusable defaults securely via TF_VAR_* **and other variables.**